LAB NO. 02

CLASSES IN C++

Lab Objectives

Following are the lab objectives:

1. Learn to define classes in C++ to represent an object
2. Learn to define setters and getters for data members
3. Learn to define constructors of class

Student ID 9864 **Student Name** Ali Raza

	Marks	Obtained Marks	Comments
Task 1	**15**		
Task 2	**15**		
Task 3	**15**		
Total Marks	**45**		

Lab Instructor

Lab Objectives and CLOs Mapping

Lab Objectives	CLOs		
	a	b	c
1			
2			
3			

Instructions

- This is individual Lab work/task.
- Complete this lab work within lab timing.
- Discussion with peers is not allowed.
- You can consult any book, notes & Internet.
- Copy paste from Internet will give you **negative marks**.
- Lab work is divided into small tasks, complete all tasks sequentially.
- Show solution of each lab task to your Lab Instructor.
- In-Lab Exercises/Tasks
- **Write your code at provided space after each question**
- **You need to upload code for all tasks at Google Class.**

OOP:

Object-Oriented Programming (OOP) in C++ is a programming approach where a program is organized around **objects** rather than functions.

- An **object** represents a real-world entity (like a *Car, Student, Bank Account*).
- Each object contains **data (attributes)** and **functions (methods)** that operate on that data.
- C++ supports OOP by allowing us to create **classes** (blueprints for objects) and then create objects from those classes.

Benefits of OOP:

Oop offers several benefits to both the program designer and the user. Object-oriented contributes to the solution of many problems associated with the development and quality of software products. The principal advantages are :

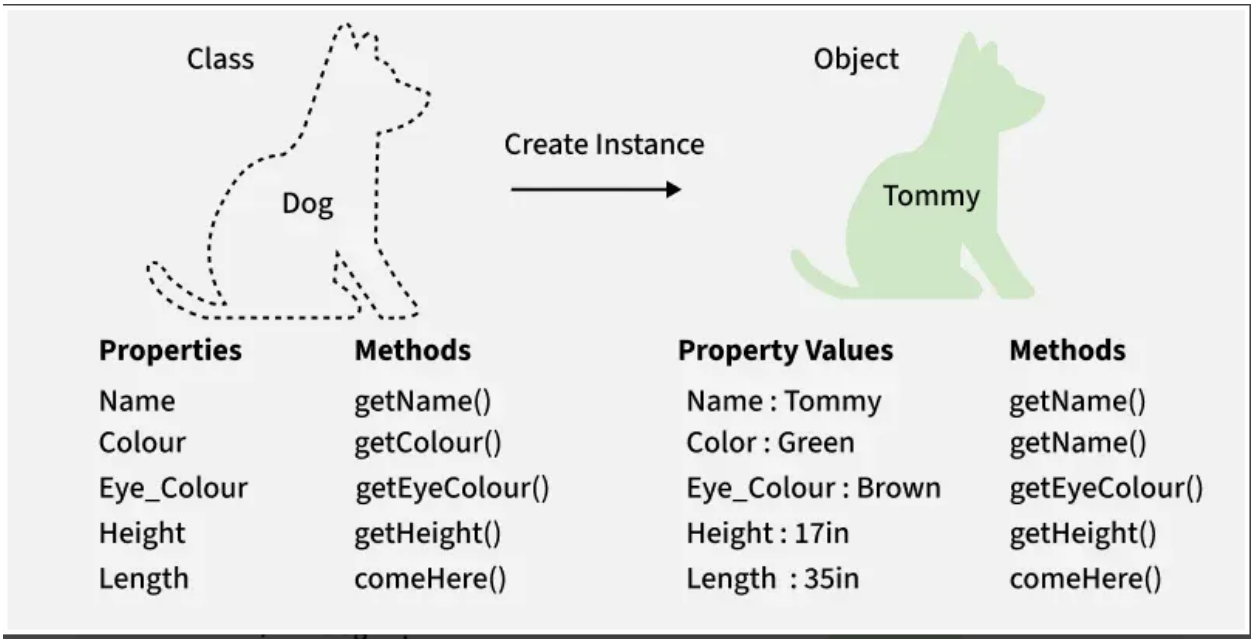
1. Through inheritance we can eliminate redundant code and extend the use of existing classes.
2. We can build programs from the standard working modules that communicate with one another, rather than having to start writing the code from scratch. This leads to saving of development time and higher productivity.
3. This principle of data hiding helps the programmer to build secure programs that can't be invaded by code in other parts of the program.
4. It is possible to have multiple instances of an object to co-exist with out any interference.
5. It is easy to partition the work in a project based on objects.
6. Object-oriented systems can be easily upgraded from small to large systems.
7. Message passing techniques for communication between objects makes the interface description with external systems much simpler.
8. Software complexity can be easily managed.

C++ Classes and Objects

In Object Oriented Programming, classes and objects are basic concepts of that are used to represent real-world concepts and entities.

Class:

- A class is a template to create objects having similar properties and behavior, or in other words.
  - A class is a blueprint for the object.
  - We can think of a class as a sketch (prototype) of a house.
  - It contains all the details about the floors, doors, windows, etc - we build the house based on these descriptions.
  - The house is the object.
- An object is an instance of a class. For example, the animal type Dog is a class, while a particular dog named Tommy is an object of the Dog class.



Classes and Objects (Here Dog is the Class and Bobby is Object)

C++ Classes

A class is a **user-defined data type,** which holds its own data members and member functions that can be accessed and used by creating an instance of that class. A C++ class is like a blueprint for an object.

Create a Class : A class must be defined before its use. C++ class is defined using the keyword **class** keyword as shown:

```
class className {
access_specifier:

   // data member

   // member method
};
```

where,

- **Data Members:** These are the variables that are defined inside the class.
- **Member Functions:** Functions declared inside a class. Also referred to as a member method.

Example:

```
#include <iostream>
using namespace std;

// Class definition
class GfG {
public:
    // Data member
    int val;

    // Member function
    void show() {
        cout << "Value: " << val << endl;
    }
};

int main() {

   // Object will declareed here
    return 0;
}
```

In the above, **GfG** class is created with a data member **val** and member function **show().** Here, member function is defined inside the class, but they can also be just declared in the class and then defined outside using **scope resolution operator ::**
The above is called class definition or template.

C++ Objects:

When a class is defined, only the specification (attributes and behaviour) for the object is defined. No memory is allocated to the class definition. To use the data and access functions defined in the class, we need to create its **objects**.

Objects are the actual entities that are created as an instance of a class. There can be as many objects of a class as desired. For example, in the above, we

discussed the class of **Cars**. If we create an actual car based on the properties of the Car class, the car we made is the object of that class.

Create Object : Once the class is defined, we can create its object in the same way we declare the variables of any other inbuilt data type.

```
className objectName;
```

This statement creates an object of **className** class.

Member Access : Members of the class can be accessed inside the class itself simply by using their assigned name.

To access them outside, the **(.) dot operator**is used with the object of the class.

```
obj.member1 // For data members
obj.member2(..) // For functions
```

There **obj** is the name of the object of the given class, **member1** is data member and **member2** is member function.

Local Class

Classes are generally declared in global scope and are accessible to every function or other classes once they are defined. But C++ also provides facility to define a class within a function. It is called **local class** in C++ and is only accessible in that function.

Nested Class

A **nested class** is a class defined within another enclosing class. As a member of the enclosing class, it has the same access rights as any other member. The members of the enclosing class do not have special access to the members of the nested class; the standard access rules apply.

This is the most common form:

```
class Outer {
public:
   class Inner {
   public:
      void display() {
         std::cout << "Inner class\n";
      }
   };
};
```

Class vs Object

The following table lists the primary differences between the classes and objects in C++:

Class	Object
A blueprint or template for creating objects.	An instance of a class with actual values.
No memory is allocated for a class until an object is created.	Memory is allocated when an object is created.
Conceptual entity describing structure and behaviour.	A real-world entity created from the class.
Defines properties and functions common to all objects of that type.	Stores specific data and manipulates it using class functions.
Represents a general concept or type.	Represents a specific instance of the class.

Implementation:

Create a Class

A class is defined in C++ using the keyword `class` followed by the name of the class.
The body of the class is defined inside curly brackets and terminated by a semicolon at the end.

```
class ClassName {
   // some data
   // some functions
};
```

For example,

```
class Room {
    public:
        double length;
        double breadth;
        double height;

        double calculate_area(){
            return length * breadth;
        }

        double calculate_volume(){
            return length * breadth * height;
        }

};
```

Here, we defined a class named `Room`.
The variables `length`, `breadth`, and `height` declared inside the class are known as **data members**.
And the functions `calculate_area()` and `calculate_volume ()` are known as **member functions** of a class.

C++ Objects

When a class is defined, only the specification for the object is defined; no memory or storage is allocated.

To use the data and access functions defined in the class, we need to create objects.

Syntax to Define Object in C++

```
ClassName object_name;
```

We can create objects of `Room` class (defined in the above example) as follows:

```
// sample function
void sample_function() {
    // create objects
    Room room1, room2;
}

int main(){
    // create objects
    Room room3, room4;
}
```

Here, two objects `room1` and `room2` of the `Room` class are created in `sample_function()`.

Similarly, the objects `room3` and `room4` are created in `main()`.

As we can see, we can create objects of a class in any function of the program.

We can also create objects of a class within the class itself or in other classes.

Also, we can create as many objects as we want from a single class.

C++ Access Data Members and Member Functions

We can access the data members and member functions of a class by using a . (dot) operator.

For example,

```
room2.calculate_area();
```

This will call the `calculate_area()` function inside the `Room` class for object `room2`.

Similarly, the data members can be accessed as:

```
room1.length = 5.5;
```

In this case, it initializes the `length` variable of `room1` to `5.5`.

Example: Object and Class in C++ Programming

```
// Program to illustrate the working of
// objects and class in C++ Programming

#include <iostream>
using namespace std;

// create a class
class Room {

   public:
    double length;
    double breadth;
    double height;

    double calculate_area() {
```

```
        return length * breadth;
    }

    double calculate_volume() {
        return length * breadth * height;
    }
};

int main() {

    // create object of Room class
    Room room1;

    // assign values to data members
    room1.length = 42.5;
    room1.breadth = 30.8;
    room1.height = 19.2;

    // calculate and display the area and volume of the room
    cout << "Area of Room =  " << room1.calculate_area() << endl;
    cout << "Volume of Room =  " << room1.calculate_volume() << endl;

    return 0;
}
```

Output

```
Area of Room =  1309
Volume of Room =  25132.8
```

In this program, we have used the `Room` class and its object `room1` to calculate the area and volume of a room.

In `main()`, we assigned the values of `length`, `breadth`, and `height` with the code:

```
room1.length = 42.5;
room1.breadth = 30.8;
room1.height = 19.2;
```

We then called the functions `calculate_area()` and `calculate_volume()` to perform the necessary calculations.

Note the use of the keyword `public` in the program. This means the members are public and can be accessed anywhere from the program.

C++ Access Specifiers

Access Specifiers

Access specifiers control how the members (attributes and methods) of a class can be accessed.

They help protect data and organize code so that only the right parts can be seen or changed.

Using Access Specifiers

The public keyword is an **access specifier.**

In the example below, the members are public - which means that they can be accessed and modified from outside the code:

Example

```
class MyClass {  // The class
  public:       // Access specifier
    // class members goes here
};
```

However, what if we want members to be private and hidden from the outside world?

In C++, there are three access specifiers:

- public - members are accessible from outside the class
- private - members cannot be accessed (or viewed) from outside the class
- protected - members cannot be accessed from outside the class, however, they can be accessed in inherited classes. You will learn more about Inheritance later.

Private

Members declared as private cannot be accessed from outside the class.

In the following example, we demonstrate the differences between public and private members:

Example

```
class MyClass {
  public:    // Public access specifier
    int x;   // Public attribute
  private:   // Private access specifier
    int y;   // Private attribute
};

int main() {
  MyClass myObj;
  myObj.x = 25;  // Allowed (public)
  myObj.y = 50;  // Not allowed (private)
  return 0;
}
```

If you try to access a private member, an error occurs:

error: y is private

LAB TASKS

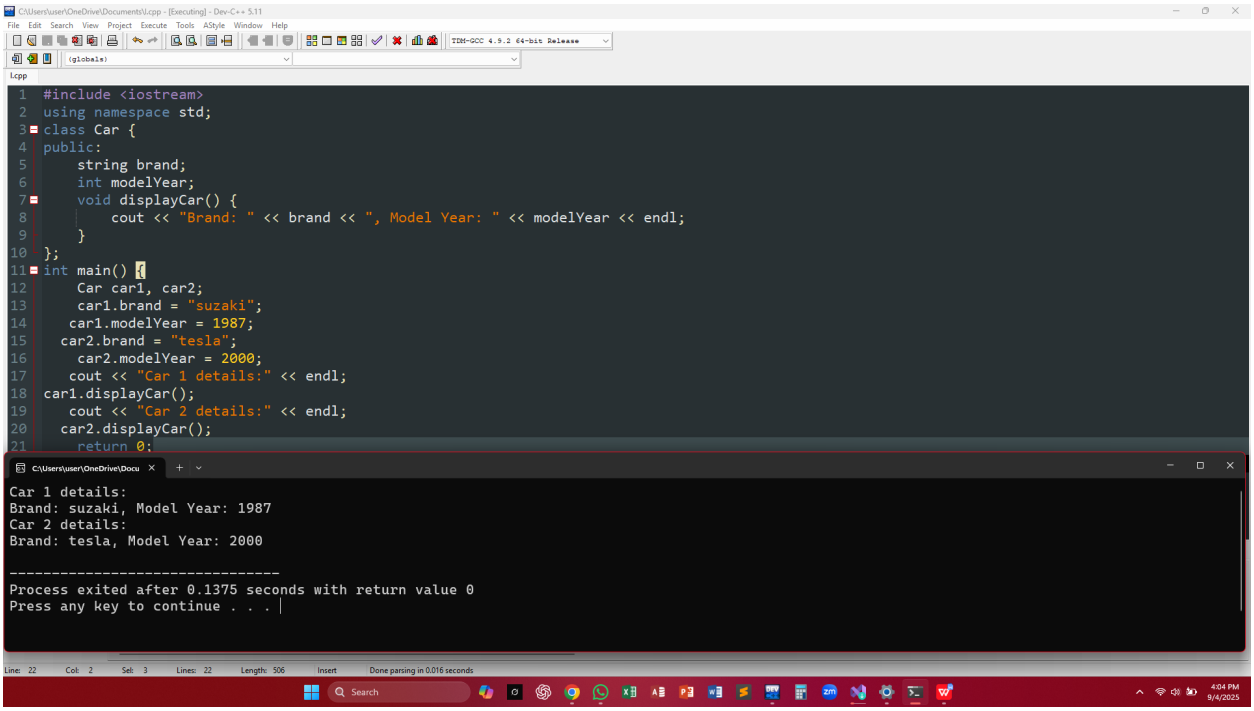
Task 1 (10 Marks)

Create a class **Car** with two public data members:

- brand (string)
- modelYear (int)

Add a function displayCar() to print the car details.
In main(), create two objects of Car and display their details.

Program:



```cpp
#include <iostream>
using namespace std;
class Car {
public:
    string brand;
    int modelYear;
    void displayCar() {
        cout << "Brand: " << brand << ", Model Year: " << modelYear << endl;
    }
};
int main() {
    Car car1, car2;
    car1.brand = "suzaki";
   car1.modelYear = 1987;
  car2.brand = "tesla";
    car2.modelYear = 2000;
   cout << "Car 1 details:" << endl;
car1.displayCar();
   cout << "Car 2 details:" << endl;
  car2.displayCar();
    return 0;
```

```
Car 1 details:
Brand: suzaki, Model Year: 1987
Car 2 details:
Brand: tesla, Model Year: 2000

--------------------------------
Process exited after 0.1375 seconds with return value 0
Press any key to continue . . .
```

Task 2

(10 Marks)

Create a class **Student** with the following:

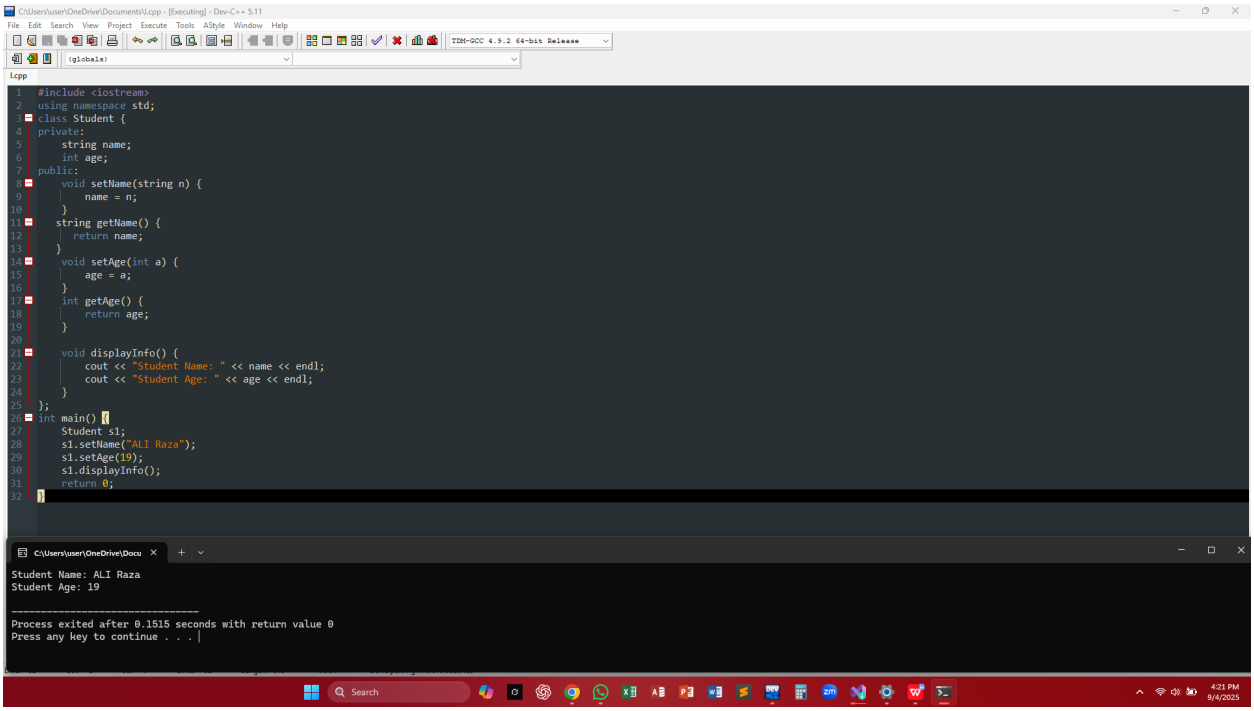
- name (string, **private**)
- age (int, **private**)

Add:

- Setters and getters for both variables.
- A function displayInfo() to print student details.

In main(), create a Student object, set values using setters, and display info.

Program:



```cpp
#include <iostream>
using namespace std;
class Student {
private:
    string name;
    int age;
public:
    void setName(string n) {
        name = n;
    }
    string getName() {
      return name;
    }
    void setAge(int a) {
        age = a;
    }
    int getAge() {
        return age;
    }

    void displayInfo() {
        cout << "Student Name: " << name << endl;
        cout << "Student Age: " << age << endl;
    }
};
int main() {
    Student s1;
    s1.setName("ALI Raza");
    s1.setAge(19);
    s1.displayInfo();
    return 0;
}
```

```
Student Name: ALI Raza
Student Age: 19

--------------------------------
Process exited after 0.1515 seconds with return value 0
Press any key to continue . . .
```

Task 3

(15 Marks)

Create a class **Room** with private members:

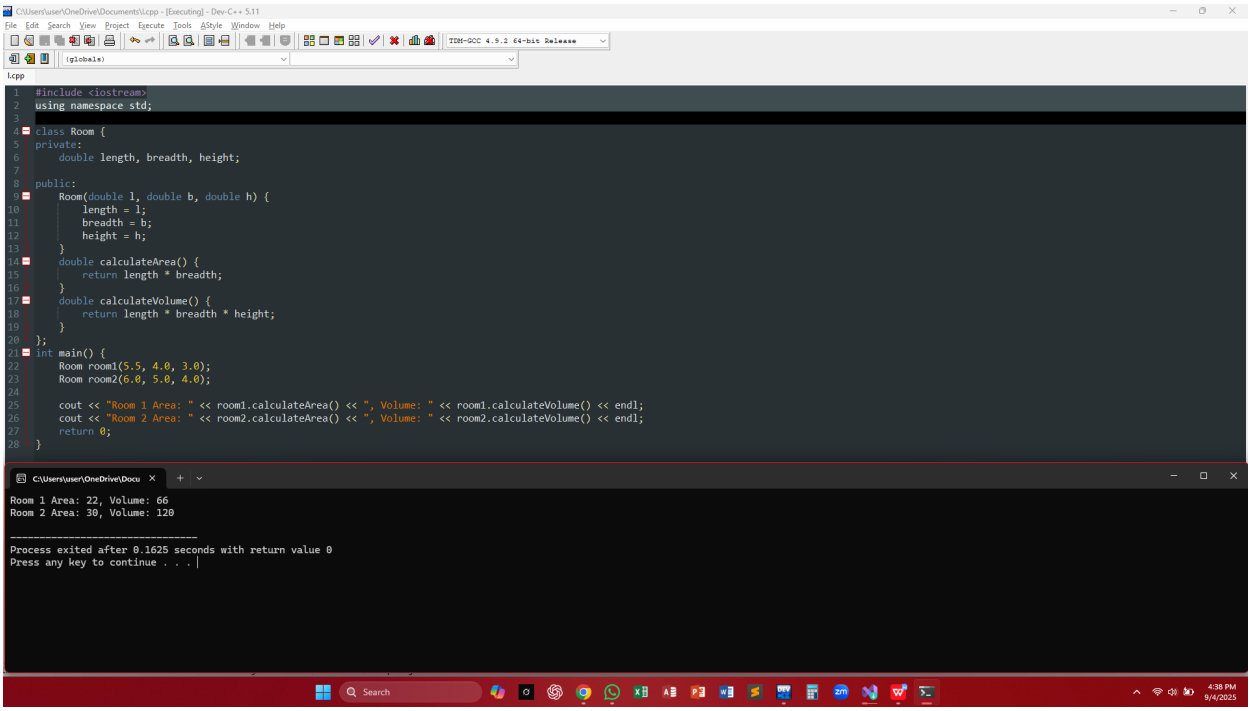
- length (double)
- breadth (double)
- height (double)

Add:

- Parameterized constructor to initialize all values.
- Function calculateArea() → returns area.
- Function calculateVolume() → returns volume.

Write a program that creates two Room objects and displays their area and volume.

Program:



```cpp
#include <iostream>
using namespace std;

class Room {
private:
    double length, breadth, height;

public:
    Room(double l, double b, double h) {
        length = l;
        breadth = b;
        height = h;
    }
    double calculateArea() {
        return length * breadth;
    }
    double calculateVolume() {
        return length * breadth * height;
    }
};
int main() {
    Room room1(5.5, 4.0, 3.0);
    Room room2(6.0, 5.0, 4.0);

    cout << "Room 1 Area: " << room1.calculateArea() << ", Volume: " << room1.calculateVolume() << endl;
    cout << "Room 2 Area: " << room2.calculateArea() << ", Volume: " << room2.calculateVolume() << endl;
    return 0;
}
```

```
Room 1 Area: 22, Volume: 66
Room 2 Area: 30, Volume: 120

--------------------------------
Process exited after 0.1625 seconds with return value 0
Press any key to continue . . .
```

Task 4

(10 Marks)

Create a class **BankAccount** with private members:

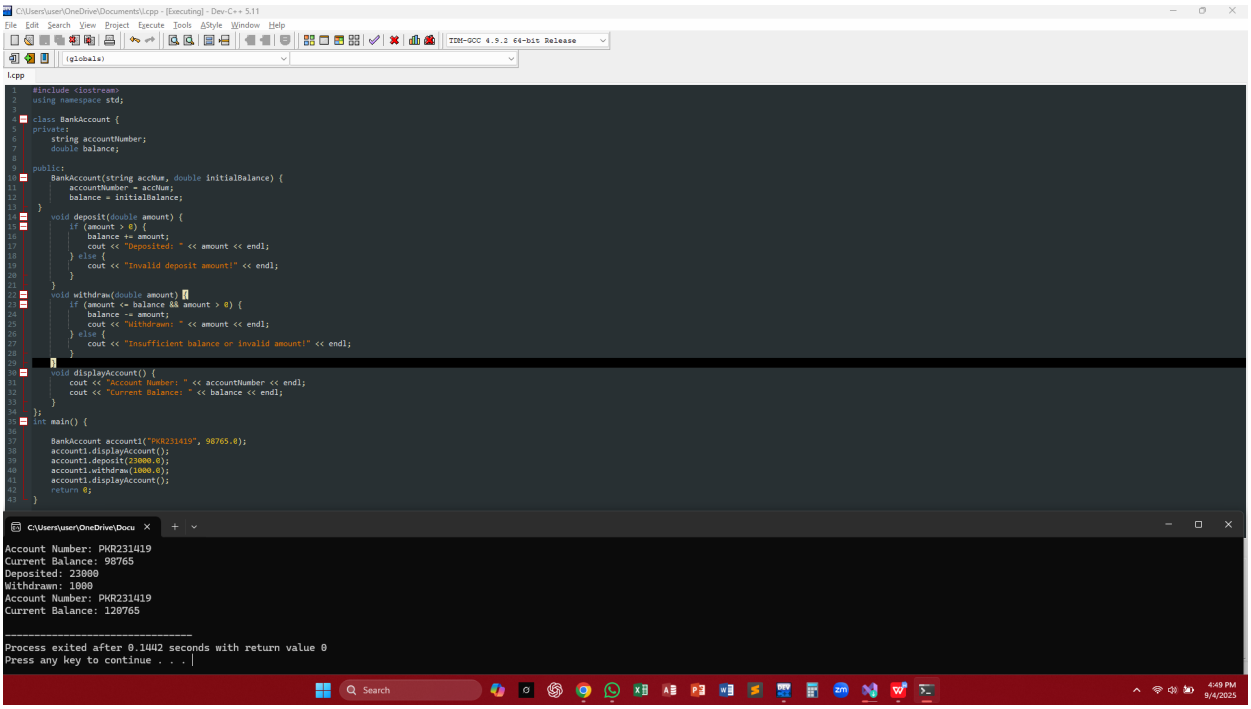
- accountNumber (string)
- balance (double)

Add:

- Constructor to initialize values.
- deposit(double amount) → add to balance.
- withdraw(double amount) → subtract from balance if sufficient.
- displayAccount() → show account details.

In main(), test deposit and withdraw operations.

Program:



Task 5 (15 mark)

Create a class called **Invoice** that a hardware store might use to represent an invoice for an item sold at the store. An Invoice should include **six** data members – a part number (type string), a part description (type string), a quantity of the item being purchased (type int), a price per item (type int) a value-added tax (VAT) rate as a decimal (type double), and a discount rate as a decimal (type double).

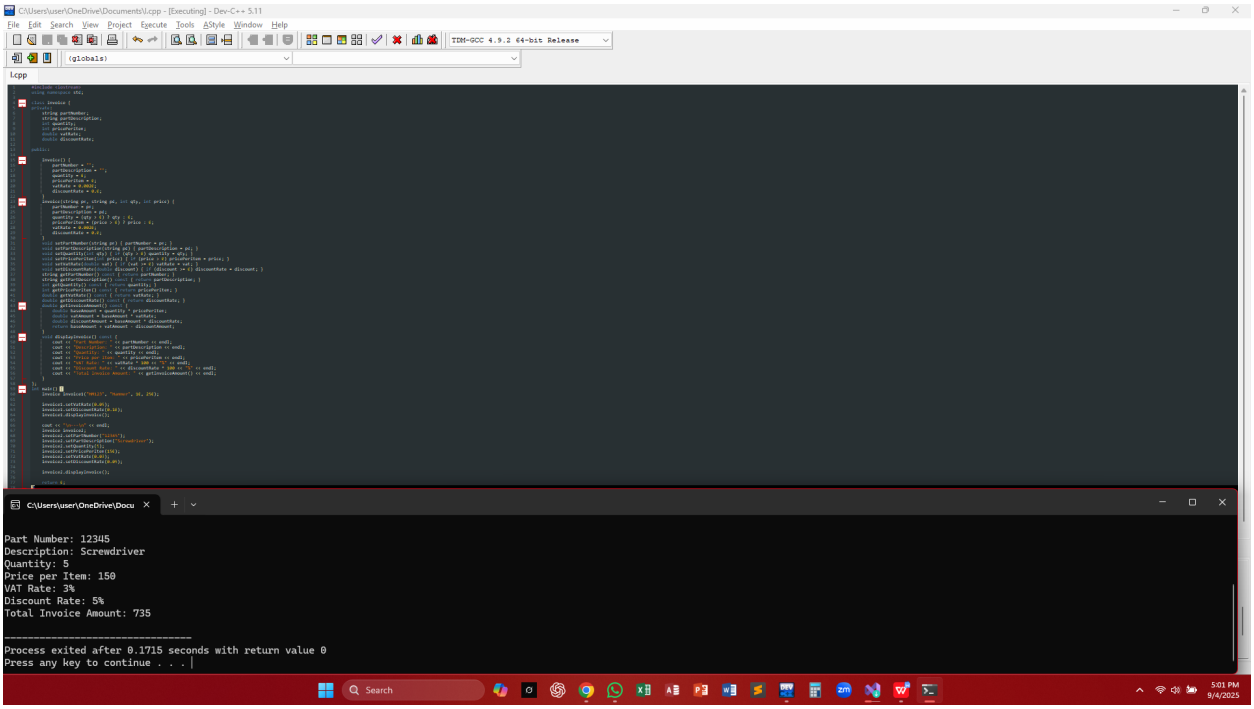
Your class should have default constructor as well as parameterized constructor that initializes the six data members. The constructor should initialize the first four data members with values from parameters and the last two data members to default values of 0.20 percent and zero respectively.

Provide setters and getters methods for all data members. In addition, provide a member function named **getInvoiceAmount( )** that calculates the invoice **amount** (i.e., multiplies the quantity by the price per item and applies the tax and discount amounts), then returns the amount. Perform

validity checks on parameters of setters, if a parameter value is not positive, it should be left unchanged.

Write a driver program to demonstrate Invoice's capabilities.

Program:



(Past your code here.)

EXTRA TASKS

Task 1

Create a class called **Distance** that includes two pieces of information as data members - feet (type int), inches (type int). Your class should have three constructors which initialize the two data members.

Provide setter and getter methods for each data member. Provide a member function **displayDistance( )** that displays the feet and inches in the form of 5'6''.

Write a test program that demonstrates class Distance's capabilities.

(Past your code here.)